

Intermediate Programming Concepts

A Sparse Matrix Processor

Dr. Gang Qu

University of Maryland, College Park
gangqu@umd.edu

1

Matrix and Sparse Matrix

- # Matrix is a set of numbers arranged in rows and columns to form a rectangular array.

$$A_{m \times n} = \{a_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}$$

1	3	8
2	-4	5

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

Row (m) → Columns (n)

- # A matrix is sparse if majority of its entries are 0.
 - Store only non-zero entries.
 - How much is majority? $O(m \times n)$

0	1	0	0
0	0	0	0
0	5	0	2
9	0	0	6
7	0	0	0

2

Matrix Operations

$$A_{m \times n} = \{a_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}, B_{m \times n} = \{b_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}$$

$$\# (A+B)_{m \times n} = \{a_{ij} + b_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}$$

$$\# (A-B)_{m \times n} = \{a_{ij} - b_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}$$

$$A_{m \times n} = \{a_{ij}\}_{1 \leq i \leq m, 1 \leq j \leq n}, B_{n \times k} = \{b_{ij}\}_{1 \leq i \leq n, 1 \leq j \leq k}$$

$$\# (A*B)_{m \times k} = \{a_{i1}b_{1j} + a_{i2}b_{2j} + \dots + a_{ik}b_{kj}\}_{1 \leq i \leq m, 1 \leq j \leq k}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -1 & 0 & 3 \end{bmatrix} \times \begin{bmatrix} 7 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 7 & 0 & 0 \\ -7 & 0 & 3 \end{bmatrix}$$



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

3

3

Find a Path in Sparse Matrix

- # In a sparse matrix, a path from entry p to entry q is a sequence of non-zero entries $\{r_0, r_1, \dots, r_i, r_{i+1}, \dots, r_n\}$ such that

- $r_0 = p$

- $r_n = q$

- r_i and r_{i+1} are in the same row or column

- The distance of a path is the sum of the distance (row/column difference) between each pair (r_i, r_{i+1}) .

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 0 & 2 \\ 9 & 0 & 0 & 6 \\ 7 & 0 & 0 & 0 \end{bmatrix}$$

- # Example:

- $A_{1,2}$ to $A_{3,4}$: distance $2+2 = 4$

- $A_{3,2}$ to $A_{5,1}$: distance $2+1+3+1 = 7$



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

4

4

About the Project

- # What will be available?
 - Project description, this set of slides
 - Q_and_A.txt, sparse.h, example sparse.c and p2.c
- # What to submit?
 - sparse_1.c and sparse_2.c
- # What are the main tasks?
 - Implement all the functions in sparse.h
 - Build the sparse matrix from "input"
 - Develop an algorithm to find a (shortest) path between two entries in the sparse matrix



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

5

5

How to Start the Project

- # project description
 - understand the high level requirements.
 - Read again and highlight the important features.
- # public/PROJECT/P2/ files
 - Copy input files and sample codes to your directory
 - Visit the Q_and_A.txt often and read it carefully.
 - You can add new functions and features to the structures in sparse.c, but not sparse.h
- # Work on the details
 - **plan data structures** and **design algorithms**.
 - Reference project description and sample code



MeshSec Lab

© Gang Qu (gangqu@umd.edu)

6

6

Separate Compilation and Linking

sparse.h: header file, don't edit.

sparse.c: your work, make sure

#include "sparse.h"

p2_1.c: has main(), don't edit

Compilation and linking

gcc -c sparse.c -o sparse.o

gcc p2_1.c sparse.o



MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

7

7

Data Structure

ENTRY

```
typedef struct entry
{
    int val; // non-zero integer value
    int row, col; // entry's position in the matrix
    struct entry *left, *right; // pointers to the first non-zero entry of the
                                // matrix above and below this entry
    struct entry *up, *down; // pointers to the first non-zero entry of the
                              // matrix to the left and right of this entry
    struct entry *next; // a pointer to the next entry in the matrix
    void *info; // you can use this pointer to store any information
                // that you want to associate with this entry
} ENTRY;
```

val
row
col
left
right
up
down
next
info

0	1	0	0
0	0	0	0
0	5	0	2
9	0	0	6
7	0	0	0



MeshSec Lab

@ Gang Qu (gangqu@umd.edu)

8

8

Data Structure

/	head
0	num
/	info

#ROOT #Input matrix

```

typedef struct root
{ ENTRY *head;

  long num;
  void *
} ROOT;
  
```

// root to a list of all non-zero entries
 // first entry in a "matrix" or a list of ENTRY's.
 // You can traverse the list with appropriate
 // "next" pointer. That is, use "next" to access
 // the entire matrix.
 // the number of non-zero entries in the matrix

1	8	7	5	6	1	9	7	9	1
2	2	5	2	3	5	5	3	3	6
2	4	3	3	5	1	2	3	1	3
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/
/	/	/	/	/	/	/	/	/	/

MeshSec Lab © Gang Qu (gangqu@umd.edu)

9

Things to Keep in Mind

- # Project description, sample executable, sample input files, Q_and_A.txt (pay attention to assumptions, error handling, etc)
- # Documentation
- # Backup
- # Incremental test and debug
- # Talk or email to Dr. Qu and the TAs
- # Start early and good luck!

MeshSec Lab © Gang Qu (gangqu@umd.edu)

10

More Examples

1	8	1
1	6	8
1	4	7
1	7	1

2	2	1
2	4	8
1	3	7
2	3	5
1	5	6

			7		8	1	1
		7					6
		1	5	8			

4
/

5
/

1	8	7	1
1	1	1	1
8	6	4	7
/	/	/	/
/	/	/	/
/	/	/	/
/	/	/	/
/	/	/	/

1	8	7	5	6
2	2	1	2	1
2	4	3	3	5
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/

val
row
col
left
right
up
down
next
info

search() and find_max() are trivial using this singly linked list (with pointer next)

11

11

More Examples

2	2	1
2	4	8
1	3	7
2	3	5
1	5	6

2	2	1
2	3	8
2	1	7
3	2	6
3	1	7
3	3	9

		7		6
	1	5	8	
7	1	8		
7	6	9		

5
/

6
/

1	8	7	5	6
2	2	1	2	1
2	4	3	3	5
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/
/	/	/	/	/

1	8	7	6	7	9
2	2	2	3	3	3
2	3	1	2	1	3
/	/	/	/	/	/
/	/	/	/	/	/
/	/	/	/	/	/
/	/	/	/	/	/
/	/	/	/	/	/

Addition/subtraction can be done with this singly linked list too, but it is not trivial.

12

12